

semana 5- aula 01

Pilares da programação orientada a objetos

Introdução à programação orientada a objetos e seus pilares

Código da aula: [SIS]ANO1C3B1S5A1

Exposição:

Paradigmas de Programação: Uma Definição

Paradigmas de programação são abordagens ou estilos fundamentais de construir a estrutura e a organização de um programa de computador. Eles representam diferentes maneiras de pensar sobre o problema a ser resolvido e de como o código deve ser escrito e executado. Cada paradigma oferece um conjunto de conceitos, princípios e técnicas que influenciam a forma como os desenvolvedores modelam problemas e implementam soluções.

A Programação Orientada a Objetos (POO) é um paradigma de programação que organiza o software em torno de "objetos" em vez de focar em funções e lógica. Um objeto pode ser visto como uma entidade que agrupa dados (atributos) e os comportamentos (métodos) que operam nesses dados. Essa abordagem busca aproximar o desenvolvimento de software da forma como percebemos o mundo real, modelando entidades complexas de forma mais intuitiva e gerenciável.

Além da Programação Orientada a Objetos, outros paradigmas de programação comuns incluem:

- **Programação Imperativa:** Foca em descrever passo a passo como o programa deve executar, alterando o estado do programa através de comandos. Exemplos incluem C e Pascal.
- **Programação Declarativa:** Foca em descrever o que o programa deve alcançar, sem especificar explicitamente como fazê-lo. Exemplos incluem SQL e Prolog.
- **Programação Funcional:** Trata a computação como a avaliação de funções matemáticas puras, evitando efeitos colaterais e mutabilidade de estado. Exemplos incluem Haskell e Lisp.
- **Programação Procedural:** Organiza o código em um conjunto de procedimentos (funções ou sub-rotinas) que realizam tarefas específicas. É uma forma de programação imperativa. Exemplo clássico é o C (em sua forma mais básica).

É importante notar que muitos projetos de software modernos podem utilizar uma abordagem multi-paradigma, combinando elementos de diferentes paradigmas para aproveitar seus respectivos benefícios em diferentes partes do sistema. A escolha

do paradigma ou da combinação de paradigmas depende das características do problema a ser resolvido, dos requisitos do projeto e das preferências da equipe de desenvolvimento.

Conceitos Fundamentais da POO:

Para entender a POO em profundidade, é crucial compreender seus quatro pilares principais:

1. Encapsulamento: Este princípio envolve agrupar os dados (atributos) e os métodos (comportamentos) que operam sobre esses dados dentro de uma única unidade, a classe. O encapsulamento também implementa o ocultamento de dados, protegendo o estado interno de um objeto de acesso direto e modificação externa não autorizada. O acesso aos dados é controlado através de métodos específicos (getters e setters), permitindo a implementação de lógica de validação ou manipulação antes que os dados sejam acessados ou modificados.

Exemplo: Considere uma classe `ContaBancaria`. Os atributos como `saldo` e `numeroConta` são encapsulados dentro da classe. O acesso e a modificação do `saldo` só podem ser feitos através de métodos como `depositar()` e `sacar()`, que podem incluir verificações de segurança (por exemplo, não permitir saques negativos).

2. Herança: A herança é um mecanismo que permite que uma classe (a subclasse ou classe filha) herde propriedades (atributos) e comportamentos (métodos) de outra classe (a superclasse ou classe pai). Isso promove a reutilização de código, pois características comuns podem ser definidas na superclasse e automaticamente disponibilizadas para suas subclasses. As subclasses também podem adicionar novos atributos e métodos ou sobrescrever os métodos herdados para fornecer um comportamento mais específico.

Exemplo: Podemos ter uma superclasse `Veiculo` com atributos como `marca`, `modelo` e métodos como `ligar()` e `desligar()`. Subclasses como `Carro` e `Moto` podem herdar esses atributos e métodos, e também definir seus próprios atributos (por exemplo, `numeroPortas` em `Carro`, `cilindrada` em `Moto`) e comportamentos específicos (por exemplo, um método `acelerarRapido()` em `Moto`).

3. Polimorfismo: Polimorfismo (que significa "muitas formas") permite que objetos de diferentes classes respondam ao mesmo método de maneiras distintas. Isso é alcançado através da sobrescrita de métodos (overriding) ou da sobrecarga de métodos (overloading).
 - Sobrescrita (Overriding): Uma subclasse fornece uma implementação específica para um método que já está definido em sua superclasse.
 - Sobrecarga (Overloading): Uma classe pode ter múltiplos métodos com o mesmo nome, mas com diferentes listas de parâmetros (em número, tipo ou ordem). O compilador ou interpretador determina qual método chamar com base nos argumentos fornecidos.
4. Exemplo: Voltando ao exemplo de `Veiculo`, tanto `Carro` quanto `Moto` podem ter um método `acelerar()`. No entanto, a implementação desse método será diferente em cada classe, refletindo a forma como cada veículo acelera. Outro exemplo seria um método `calcularArea()` em uma classe `Forma`. Subclasses como `Retangulo` e `Circulo` implementariam `calcularArea()` de maneiras diferentes, com base em suas respectivas fórmulas.
5. Abstração: A abstração envolve simplificar a complexidade do mundo real, modelando apenas os aspectos essenciais de um objeto que são relevantes para o contexto em questão. Ela se concentra no "o quê" um objeto faz, em vez de "como" ele faz. Classes abstratas e interfaces são mecanismos comuns para implementar a abstração.
 - Classes Abstratas: Classes que não podem ser instanciadas diretamente e podem conter métodos abstratos (métodos sem implementação). Subclasses concretas devem fornecer implementações para esses métodos abstratos.
 - Interfaces: Contratos que definem um conjunto de métodos que uma classe deve implementar. Uma classe pode implementar múltiplas interfaces.
6. Exemplo: Podemos definir uma classe abstrata `Animal` com um método abstrato `emitirSom()`. Subclasses concretas como `Cachorro` e `Gato` seriam obrigadas a implementar o método `emitirSom()` com seus respectivos sons ("Au Au" e "Miau"). A abstração nos permite trabalhar com objetos `Animal` de forma genérica, sabendo que todos eles podem emitir um som, sem nos preocuparmos com os detalhes específicos de cada animal.
- Reusabilidade de Código: A herança permite que novas classes sejam construídas a partir de classes existentes, economizando tempo e esforço de desenvolvimento.
- Modularidade: A POO organiza o código em unidades independentes (objetos), tornando-o mais fácil de entender, depurar e manter.
- Flexibilidade: O polimorfismo permite que o código se adapte a diferentes tipos de objetos de forma transparente.

- Manutenibilidade: A estrutura orientada a objetos tende a produzir um código mais organizado e fácil de modificar e estender.
- Modelagem do Mundo Real: A POO facilita a modelagem de sistemas complexos, pois os objetos podem representar entidades do mundo real e suas interações.

Exemplo Prático Simplificado em Python:

Python



```
class Animal:
    def __init__(self, nome):
        self.nome = nome

    def emitir_som(self):
        print("Som genérico de animal")

class Cachorro(Animal):
    def emitir_som(self):
        print("Au Au!")

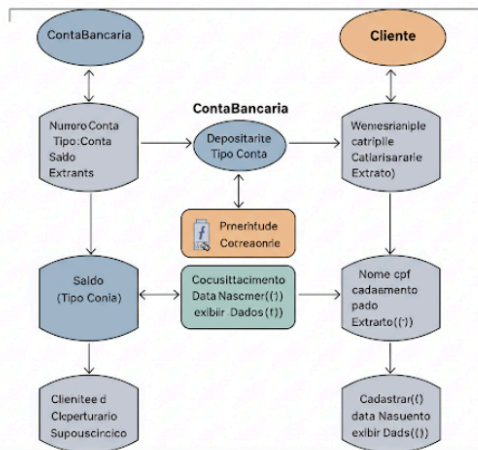
class Gato(Animal):
    def emitir_som(self):
        print("Miau!")

# Criando objetos
cachorro = Cachorro("Rex")
gato = Gato("Mingau")

# Polimorfismo em ação
animais = [cachorro, gato]
for animal in animais:
    print(f"{animal.nome} faz:")
    animal.emitir_som()
```

Neste exemplo, `Animal` é a superclasse, e `Cachorro` e `Gato` são subclasses que herdam de `Animal`. Ambas as subclasses sobrescrevem o método `emitir_som()` para fornecer seus próprios comportamentos específicos, demonstrando o polimorfismo.

Exemplo de um programa usando POO:



Diagrama

O Diagrama de Classes UML para este exemplo seria representado da seguinte forma:

Snippet de código

```

@startuml
class ContaBancaria {
    - numeroConta: String
    - saldo: Double
    - titular: Cliente
    + depositar(valor: Double): void
    + sacar(valor: Double): boolean
    + obterSaldo(): Double
    + obterNumeroConta(): String
}

class Cliente {
    - nome: String
    - cpf: String
    + obterNome(): String
    + obterCpf(): String
}

ContaBancaria --* Cliente : titular

@enduml

```

Vídeos:

Paradigmas de Programação

<https://youtu.be/EefVmQ2wPIM?si=5yfUf67o5VCr8A73>

Paradigmas de Programação // Vlog #26



Código F...
718 mil inscritos

Seja membro

Inscrito

6,1 mil



Compartilhar



Programação Orientada a Objeto

<https://youtu.be/QY0Kdg83orY?si=Vb6vBlrgzSRuK-cK>

Programação Orientada a Objetos (POO) // Dicionário do Programador



Código F...
718 mil inscritos

Seja membro

Inscrito

24 mil



Compartilhar



semana 5- aula 02

Pilares da programação orientada a objetos

Introdução à programação orientada a objetos e seus pilares

Código da aula: [SIS]ANO1C3B1S5A2

Objetivos da Aula:

- ❖ Conhecer os fundamentos da programação orientada a objetos (POO) no contexto de seus conceitos e aplicações.
- ❖ Prestar apoio técnico na elaboração da documentação de sistemas. Conhecer frameworks de desenvolvimento ágeis, utilizando tecnologias de CI e CD que trabalham em conjunto com a segurança do ambiente funcional e as entregas divididas em partes que agregam valor ao negócio de forma rápida. Trabalhar a resolução de problemas computacionais.
- ❖ Recurso audiovisual para exibição de vídeos e imagens;
- ❖ Caderno para anotações. Recursos didáticos Competências da unidade (técnicas e socioemocionais)

Exposição:

Slide 5

Classe

É uma estrutura fundamental na POO que serve como um molde para criar algo mais concreto: os objetos. Pense na classe como uma receita de bolo. A receita detalha os ingredientes e passos necessários para fazer o bolo, mas não é o bolo em si. Da mesma forma, uma classe detalha as características e os comportamentos (por meio de atributos e métodos) que os objetos derivados dela

terão, mas não é o objeto. Por exemplo, uma classe "Automóvel" define que todo automóvel tem uma marca, um modelo, uma cor e uma função para "acelerar" ou "frear". No entanto, a classe "Automóvel" em si não é um carro que você pode dirigir; ela apenas descreve o que um carro é e faz.

Slide 6

Objeto

São as entidades criadas usando classes como modelo. Continuando com a analogia da receita, um objeto é o bolo que você faz usando a receita. Cada bolo pode ter sabores variados ou decorações diferentes, mas todos seguem a base da receita. No caso da classe "Automóvel", um objeto seria um carro específico, como um Ford Mustang vermelho de 2020. Esse carro específico tem características concretas (atributos como marca, modelo e cor) e pode realizar ações específicas (comportamentos definidos pelos métodos como acelerar e frear).

Slide 7

Métodos

São como as instruções na receita que dizem o que fazer com os ingredientes.

Em uma classe, métodos definem ações ou comportamentos que os objetos podem executar. Eles são como funções que operam dentro de uma classe e muitas vezes manipulam os atributos dessa classe. Por exemplo, um método em uma classe "Automóvel" pode ser "ligarMotor", que muda o estado do carro de "motor desligado" para "motor ligado". Outro método pode ser "exibirDetalhes", que fornece informações sobre o carro, como a marca, o modelo e a cor.

Exemplo

O Diagrama de Classes UML para este exemplo seria representado da seguinte forma:

Snippet de código



```
@startuml
class ContaBancaria {
    - numeroConta: String
    - saldo: Double
    - titular: Cliente
    + depositar(valor: Double): void
    + sacar(valor: Double): boolean
    + obterSaldo(): Double
    + obterNumeroConta(): String
}

class Cliente {
    - nome: String
    - cpf: String
    + obterNome(): String
    + obterCpf(): String
}

ContaBancaria --* Cliente : titular

@enduml
```

Vamos abordar um problema de segurança comum em sistemas onde dados sensíveis precisam ser protegidos.

O **encapsulamento** é um princípio fundamental da programação orientada a objetos (POO) que nos ajuda a proteger dados sensíveis. Ele agrupa dados (atributos) e os métodos que operam sobre esses dados dentro de uma única unidade (uma classe), restringindo o acesso direto aos atributos e controlando como eles podem ser modificados ou acessados através de métodos públicos.

Passo 1: Exemplo de Código Python com Encapsulamento

Considere um cenário onde temos uma classe `Usuario` que precisa armazenar um nome de usuário, uma senha e um token de API. Queremos garantir que a senha e

o token não sejam acessados diretamente e que qualquer modificação neles passe por validação ou operações seguras.

```
import hashlib
```

```
class Usuario:
```

```
    def __init__(self, username, password, api_token):
```

```
        # Atributos "privados" (por convenção, com prefixo __)
```

```
        self.__username = username
```

```
        self.__password_hash = self.__hash_password(password) # Armazena o hash da senha
```

```
        self.__api_token = api_token # Este poderia ser criptografado na vida real
```

```
    def __hash_password(self, password):
```

```
        """Método privado para gerar o hash da senha."""
```

```
        return hashlib.sha256(password.encode()).hexdigest()
```

```
    def verificar_senha(self, password):
```

```
        """
```

```
        Método público para verificar a senha.
```

```
        Não expõe o hash da senha diretamente.
```

```
        """
```

```
        return self.__hash_password(password) == self.__password_hash
```

```
    def get_username(self):
```

```
        """
```

```
        Método público para acessar o nome de usuário.
```

```

"""

return self.__username


def get_api_token(self, auth_key):
    """
    Método público para acessar o token da API.

    Requer uma chave de autenticação para simular acesso controlado.
    """

    if auth_key == "chave_secreta_de_acesso": # Exemplo simples, na vida real
seria mais complexo

        return self.__api_token

    else:

        print("Acesso negado ao token da API.")

        return None


def alterar_senha(self, current_password, new_password):
    """
    Método público para alterar a senha.

    Requer a senha atual para validação antes de permitir a alteração.
    """

    if self.verificar_senha(current_password):

        self.__password_hash = self.__hash_password(new_password)

        print("Senha alterada com sucesso!")

        return True

    else:

```

```
print("Erro: Senha atual incorreta.")  
  
return False
```

Exemplo de uso:

```
print("--- Teste de Encapsulamento ---")  
  
user1 = Usuario("alice.s", "senha123", "token_xyz_123")
```

```
print(f"Nome de usuário: {user1.get_username()}")
```

Tentativa de acessar atributos privados diretamente (irá falhar ou retornar AttributeError)

try:

```
    print(f"Senha hash (tentativa direta): {user1.__password_hash}")
```

except AttributeError as e:

```
    print(f"Erro ao tentar acessar atributo privado diretamente: {e}")
```

Verificando a senha através do método público

```
print(f"Verificando senha 'senha123': {user1.verificar_senha('senha123')}")
```

```
print(f"Verificando senha 'senha_errada': {user1.verificar_senha('senha_errada')}")
```

Acessando o token da API com e sem a chave de autenticação

```
print(f"Token da API (com chave correta):  
{user1.get_api_token('chave_secreta_de_acesso')}")
```

```
print(f"Token da API (com chave incorreta):  
{user1.get_api_token('chave_incorreta')}")
```

Alterando a senha

```
user1.alterar_senha("senha123", "nova_senha_forte")
```

```
print(f"Verificando nova senha 'nova_senha_forte':  
{user1.verificar_senha('nova_senha_forte')}")
```

```
print(f"Verificando senha antiga 'senha123': {user1.verificar_senha('senha123')}")
```

Tentativa de alterar a senha com senha atual incorreta

```
user1.alterar_senha("senha_errada", "outra_senha")
```

Neste exemplo:

- Os atributos `__username`, `__password_hash` e `__api_token` são "privados" por convenção, indicados pelo prefixo `__`. Isso significa que eles não devem ser acessados diretamente de fora da classe.
- Os métodos como `__hash_password` são internos à classe e ajudam a gerenciar a lógica de hashing da senha, sem expor os detalhes.
- Métodos **públicos** como `get_username()`, `verificar_senha()`, `get_api_token()` e `alterar_senha()` fornecem uma interface controlada para interagir com os dados sensíveis.
- Para acessar o `api_token`, introduzimos uma verificação (`auth_key`), simulando um controle de acesso mais rigoroso, um exemplo prático de como o encapsulamento pode ser usado para proteger dados.
- A alteração da senha só é permitida se a senha atual for verificada corretamente, evitando que um atacante que consiga acesso ao objeto mude a senha sem o conhecimento da senha original.

Representação em Diagrama UML

Aqui está um diagrama de classes UML que representa a classe `Usuario` e o encapsulamento aplicado:



```
@startuml
class Usuario {
    - __username: str
    - __password_hash: str
    - __api_token: str
    --
    + __init__(username: str, password: str, api_token: str)
    - __hash_password(password: str): str
    + verificar_senha(password: str): bool
    + get_username(): str
    + get_api_token(auth_key: str): str
    + alterar_senha(current_password: str, new_password: str): bool
}
@enduml
```

Explicação do Diagrama UML:

- **Usuario**: Nome da classe.
- **- __username: str**: O sinal de menos (-) indica que o atributo `__username` é **privado** (ou, mais precisamente, protegido pelo encapsulamento em Python, que o torna mais difícil de acessar diretamente). Ele armazena o nome de usuário como uma string.
- **- __password_hash: str**: Atributo privado para armazenar o hash da senha.
- **- __api_token: str**: Atributo privado para armazenar o token da API.
- **+ __init__(username: str, password: str, api_token: str)**: O sinal de mais (+) indica que o método é **público**. Este é o construtor da classe.
- **- __hash_password(password: str): str**: Método privado para gerar o hash da senha.
- **+ verificar_senha(password: str): bool**: Método público que verifica se a senha fornecida corresponde ao hash armazenado. Retorna um booleano.
- **+ get_username(): str**: Método público para obter o nome de usuário.
- **+ get_api_token(auth_key: str): str**: Método público para obter o token da API, exigindo uma chave de autenticação.
- **+ alterar_senha(current_password: str, new_password: str): bool**: Método público para alterar a senha, exigindo a senha atual para validação.

Este diagrama ilustra claramente como os dados sensíveis (`__password_hash`, `__api_token`) são encapsulados dentro da classe `Usuario` e só podem ser manipulados através de métodos públicos controlados, aumentando a segurança do sistema.

semana 9- aula 01

Diagramas UML

Diagrama de casos de uso

Código da aula: [SIS]ANO1C3B2S9A1

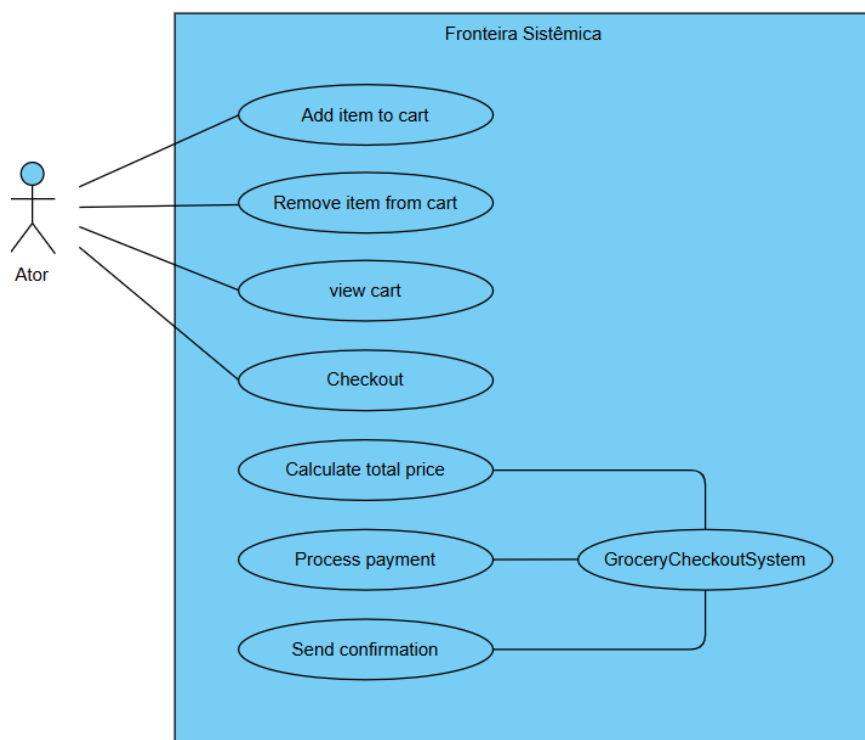
Objetivos da Aula:

- ❖ Conhecer os conceitos sobre diagramas de casos de uso (UML) de acordo com o padrão, para a visualização e a documentação de software.
- ❖ Prestar apoio técnico na elaboração da documentação de sistemas;
- ❖ Migrar sistemas, implementando rotinas e estruturas de dados mais eficazes;
- ❖ Explorar a criatividade na resolução de problemas computacionais.

Exposição:

O Diagrama de Casos de Uso é uma ferramenta visual da UML (Unified Modeling Language) utilizada na engenharia de software e análise de sistemas. **Ele descreve a funcionalidade de um sistema do ponto de vista de um usuário externo (chamado de "ator"), esses diagramas mostram a interação entre os atores e os casos de uso, que são as funcionalidades ou os processos específicos do sistema.** Em essência, ele mostra:

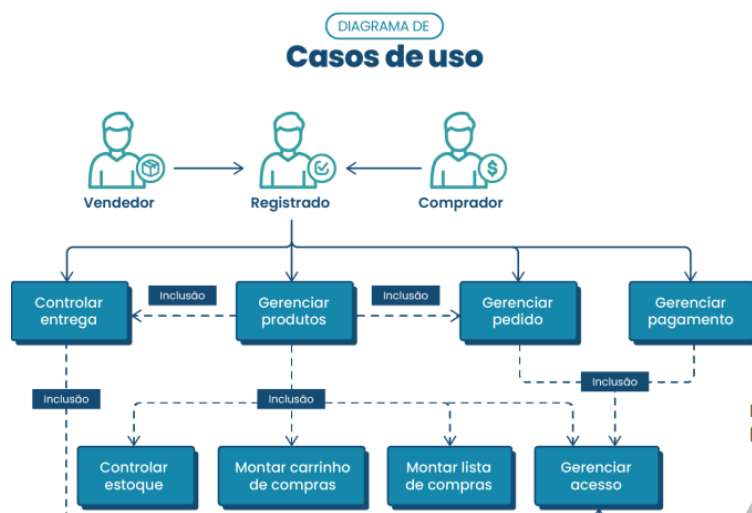
1. Quem interage com o sistema (os atores).
2. O que esses atores podem fazer com o sistema (os casos de uso).
3. Como essas funcionalidades se relacionam.



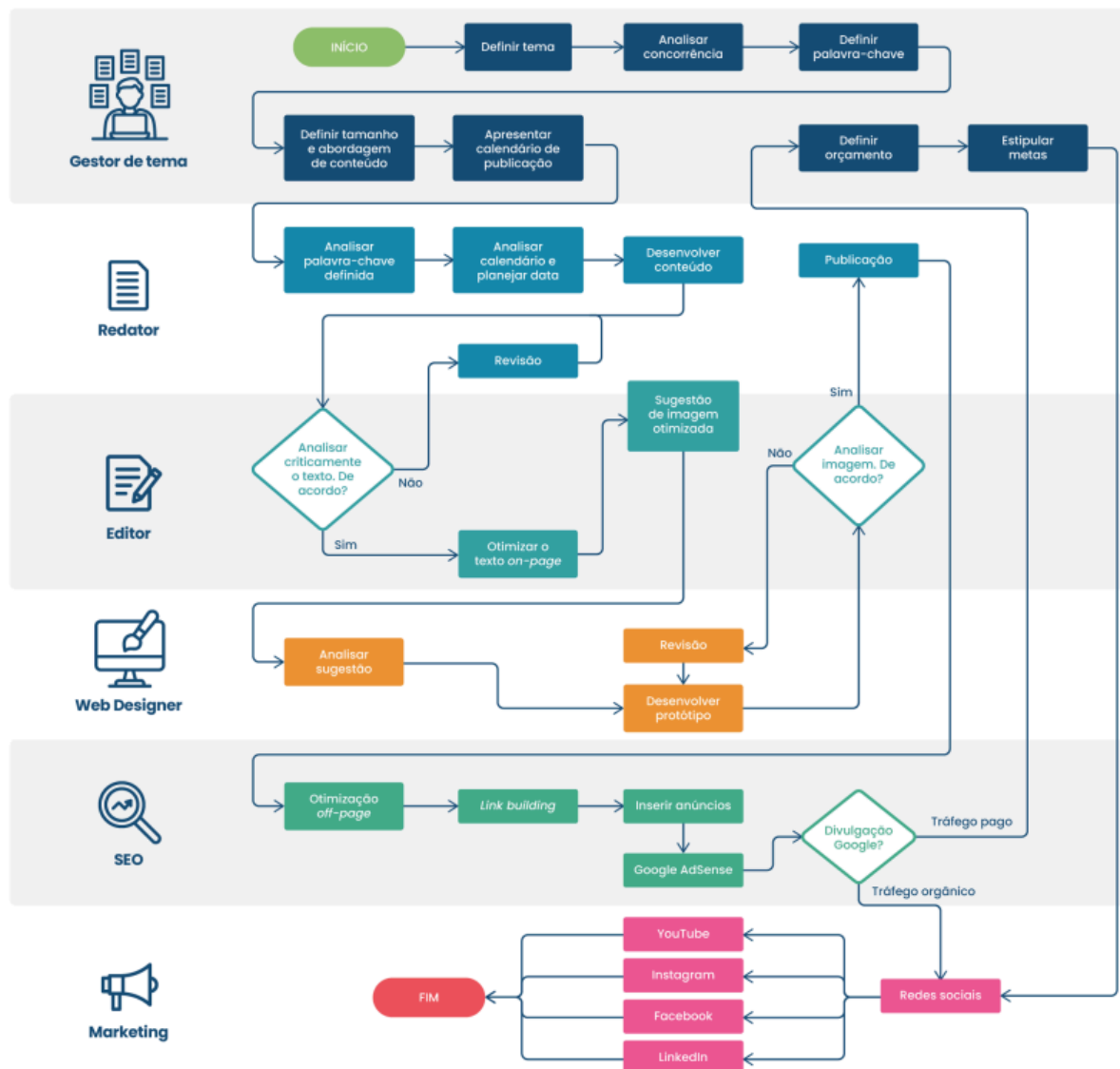
O diagrama foca no *o quê* o sistema faz (requisitos funcionais), e não no *como* ele faz (detalhes de implementação). É ótimo para entender o escopo do sistema e comunicar suas funcionalidades principais para diferentes públicos (clientes, desenvolvedores, testadores).

Componentes Principais:

1. Ator (Actor): Representa um papel desempenhado por um usuário humano, outro sistema ou hardware que interage com o sistema. É simbolizado por um boneco palito ("stick figure").
2. Caso de Uso (Use Case): Representa uma funcionalidade específica que o sistema oferece para agregar valor a um ator. É simbolizado por uma elipse contendo um verbo ou frase verbal (ex: "Fazer Login", "Consultar Saldo").
3. Sistema (System Boundary): Um retângulo que delimita o escopo do sistema, contendo os casos de uso. Os atores ficam fora desse retângulo.
4. Relacionamentos: Linhas que conectam os elementos:
 - Associação: Linha sólida entre um ator e um caso de uso, indicando que o ator participa desse caso de uso.
 - Inclusão (<<include>>): Linha tracejada com seta apontando para o caso de uso *incluído*. Indica que um caso de uso base *obrigatoriamente* incorpora a funcionalidade de outro caso de uso. (Ex: "Sacar Dinheiro" *inclui* "Validar Cartão").
 - Extensão (<<extend>>): Linha tracejada com seta apontando para o caso de uso *base*. Indica que um caso de uso pode, *opcionalmente* e sob certas condições, estender a funcionalidade de outro. (Ex: "Fazer Pedido" pode ser *estendido* por "Aplicar Cupom de Desconto").
 - Generalização: Linha sólida com uma seta de ponta vazada. Usada entre atores (indicando que um ator é um tipo especializado de outro, ex: "Cliente VIP" é um "Cliente") ou entre casos de uso (menos comum, indicando herança de comportamento).



A imagem abaixo representa um diagrama de caso de uso para um sistema de gerenciamento de conteúdo (CMS). Os relacionamentos também são visíveis: Podemos ver que o processo segue um modelo de fluxograma, trabalhando por etapas e tomadas de decisão que direcionam o fluxo, de acordo com as tarefas necessárias no momento. Importante destacar os diferentes papéis que fazem parte do processo, como Gestor de Tema, Redator, Editor, Web Designer, SEO e Marketing.



Fonte: Elaborado especialmente para o curso.

Relacionamentos são as conexões entre atores e casos de uso ou entre diferentes casos de uso. Os principais tipos de relacionamentos são:

✓ Associação: Uma linha simples conectando um ator a um caso de uso, indicando interação;

✓ Inclusão: Um relacionamento em que um caso de uso inclui a funcionalidade de outro, indicando que um caso de uso não pode ocorrer sem o outro;

✓ Extensão: Um relacionamento em que um caso de uso estende outro com comportamento adicional, muitas vezes condicional.

Exemplo abaixo:

